

EECS 489 - WN 25

Discussion 2

Assignment 1
Performance Metrics
HTTP

Outline

- Assignment 1
- Performance Metrics
- HTTP

Assignment 1

Assignment 1

Due: Wednesday, January 29 @ 11:59 p.m.

- Less than a week left
- **No late days for P1!!!**

Assignment 1

- **Part 1: Mininet Tutorial**
 - Allows you to simulate a network locally
 - We will use mininet for all four assignments, so get familiar now!
- **Part 2: iPerfer**
 - Write iPerfer tool to measure bandwidth between two hosts.
- **Part 3: Experiments**
 - Run some experiments with a given network topology and explain the results qualitatively (hand-graded).
- **Part 4: Create custom mininet topology**
 - Very simple; just modify the existing topology we give you

What is Mininet?

Simulated network with custom topology (links and hosts) that is run locally!

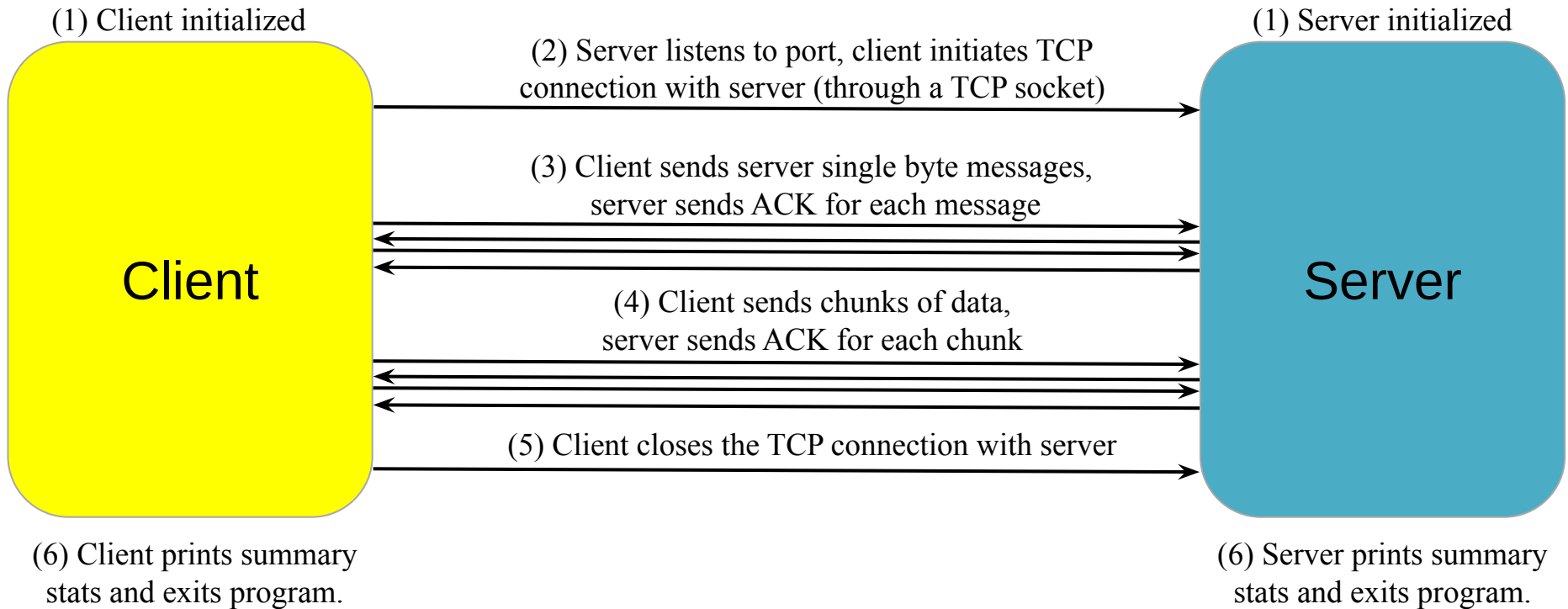
All hosts share same files (so you can run the same code from different hosts), but they appear to be completely independent machines with unique IP addresses.

Useful commands:

- `nodes` (show all hosts)
- `dump` (show all info)
- `net` (show all interfaces)
- `h1 bash` (enter terminal inside of host h1)
- `h1 ping h2` (ping h2 from h1)
- `ifconfig` (check own IP address inside of a host terminal)
- `mnbash h1` (open terminal for specific node)

<https://mininet.org/>

iPerfer: Protocol



Assignment 1: Tips & Tricks

- Server and client are the same executable, just with slightly different command-line flags.
- Feel free to use sockets code from Discussion 1
- Use `cxxopts` and `spdlog` — makes life easier!
- Verify your CMake configuration builds the executable correctly
- Make sure you are using `send` and `recv` correctly!
 - What do they return, and when do they stop blocking?

Performance Metrics

Latency and Throughput

Latency: How much time does it take for a single bit of information to travel from Host A to Host B?

- Often measured by $RTT/2$ on average
- RTT: Round-trip time
- Ideal latency equals **propagation delay**
- **We want this to be as small as possible**

Throughput: How much data (per unit of time) can be transferred between Host A and Host B?

- $\text{Throughput} = (\text{Amount of Data Transferred}) / (\text{Time to Transfer Data})$
- Ideal throughput equals **bandwidth**
- **We want this to be as large as possible**

What is an example of something that is high latency, high throughput?

What about low latency, low throughput?

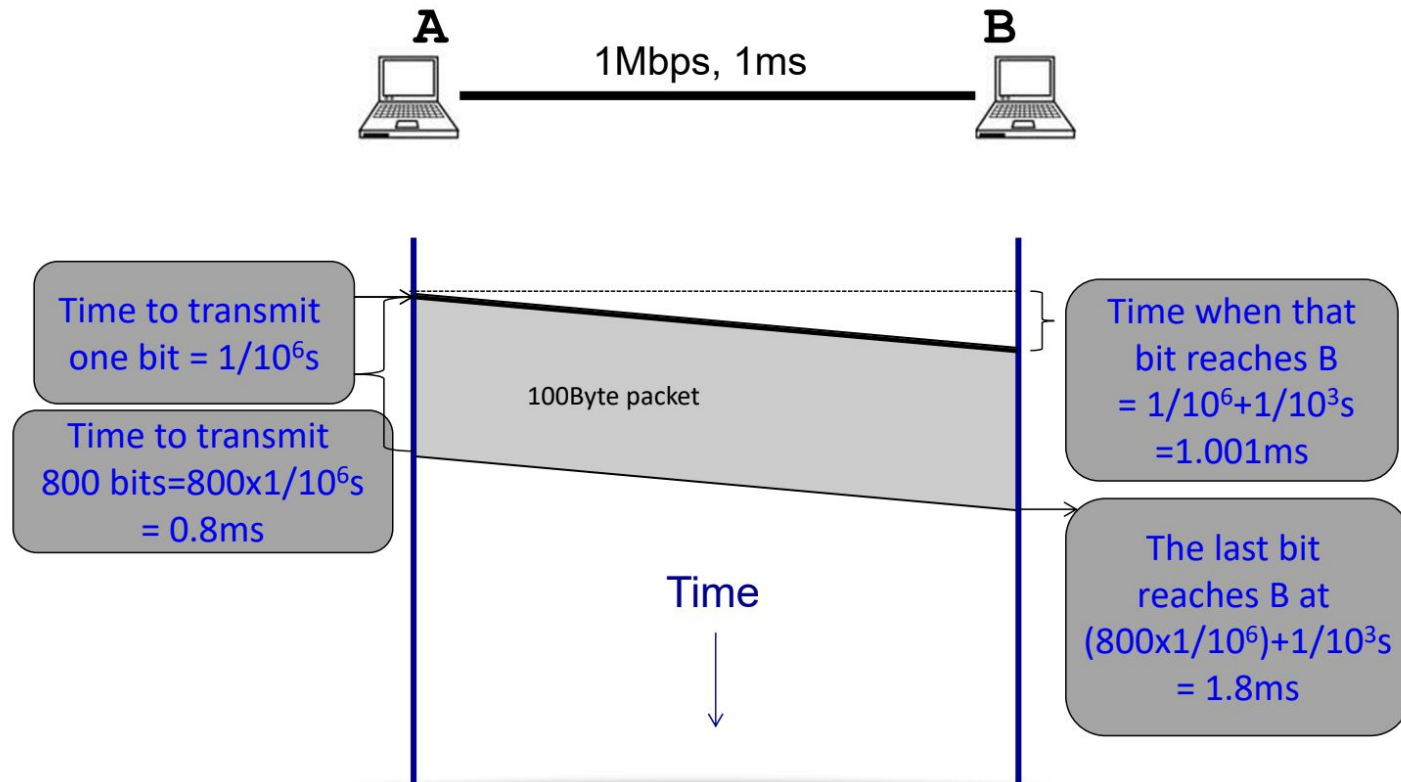
Measuring Time to Transfer Data

$$\begin{aligned}\text{Time to Transfer Data} = & \text{Transmission Delay} \\ & + \text{Propagation Delay} \\ & + \text{Queueing Delay} \\ & + \text{Processing Delay}\end{aligned}$$

- **Transmission Delay:** How long does it take to transfer all data into a link?
 - This is a one-time cost for every data transfer, and is an inherent property of the link.
- **Propagation Delay:** How long does it take for one bit to cross the link?
 - This is just the latency of the link.
- **Queueing Delay:** How long does the data spend in queues waiting to be processed by intermediate routers?
 - Not always negligible, but we can ignore for now
- **Processing Delay:** How much time is spent processing the packet for forwarding?
 - Usually negligible

Data Transfer Example

Packet delay - Sending a 100-byte packet



Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light = $3 * 10^8$ m/s.

(a) Calculate the minimum RTT (Round-Trip Time) for the link.

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light = $3 * 10^8$ m/s.

(a) Calculate the minimum RTT (Round-Trip Time) for the link.

$$\text{Propagation Delay} = 385,000 \text{ km} / (3 * 10^5 \text{ km/s}) = \mathbf{1.283 \text{ seconds}}$$

$$\text{RTT} = 2 * \text{propagation delay} = 2 * 1.28333 = \mathbf{2.567 \text{ seconds}}$$

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light = 3×10^8 m/s.

(b) Suppose Mission Control on Earth wishes to download a 25MB image from a camera on the lunar base. What is the minimum amount of time that will elapse between when the request for the data goes out and the transfer is finished? What is the throughput of the download? Assume there is no queueing or processing delay.

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light $= 3 * 10^8$ m/s.

(b) Suppose Mission Control on Earth wishes to download a 25MB image from a camera on the lunar base. What is the minimum amount of time that will elapse between when the request for the data goes out and the transfer is finished? What is the throughput of the download? Assume there is no queueing or processing delay.

$$\begin{aligned}\text{Min Time Elapsed} &= (\text{Time for request to reach moon}) + (\text{Time for moon to send back 25MB of data}) \\ &= \text{RTT}/2 + \text{RTT}/2 + \text{Transmission Delay} \\ &= \text{RTT} + (25 \text{ MB} / 100\text{Mbps}) \\ &= 2.567\text{s} + (200 \text{ Mb} / 100 \text{ Mbps}) \\ &= \mathbf{4.567 \text{ seconds}}\end{aligned}$$

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light $= 3 * 10^8$ m/s.

(b) Suppose Mission Control on Earth wishes to download a 25MB image from a camera on the lunar base. What is the minimum amount of time that will elapse between when the request for the data goes out and the transfer is finished? What is the throughput of the download, in Mbps? Assume there is no queueing or processing delay.

$$\begin{aligned}\text{Throughput} &= (25 \text{ MB}) / (\text{Propagation Delay} + \text{Transmission Delay}) \\ &= (200 \text{ Mb}) / (1.283\text{s} + 2\text{s}) \\ &= \mathbf{60.91 \text{ Mbps}}\end{aligned}$$

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light = $3 * 10^8$ m/s.

(c) What is the maximum number of bits in-flight at any given time on the link?

Performance Metrics - Q1

Suppose a 100-Mbps point-to-point link is being set up between Earth and a new lunar colony. The distance from the moon to Earth is approximately 385,000 km, and data travels over the link at the speed of light = $3 * 10^8$ m/s.

(c) What is the maximum number of bits in-flight at any given time on the link?

$$\begin{aligned}\text{Bits-in-Flight} &= \text{Bandwidth} * \text{Propagation Delay} \\ &= (100\text{Mbps}) * (1.283\text{s}) \\ &= \mathbf{1.283 * 10^8 \text{ seconds}}\end{aligned}$$

Performance Metrics - Q2

Calculate the time from first bit sent to last bit received for the following:

(a) You are sending a 5 Kb packet over a 10 Mbps link. There is a **store-and-forward switch** in the middle of the link, which begins retransmitting the packet as soon as it has finished receiving the packet. Assume each section of the link (source $\rightarrow$ switch, switch $\rightarrow$ destination) introduces a propagation delay of 100 microseconds.

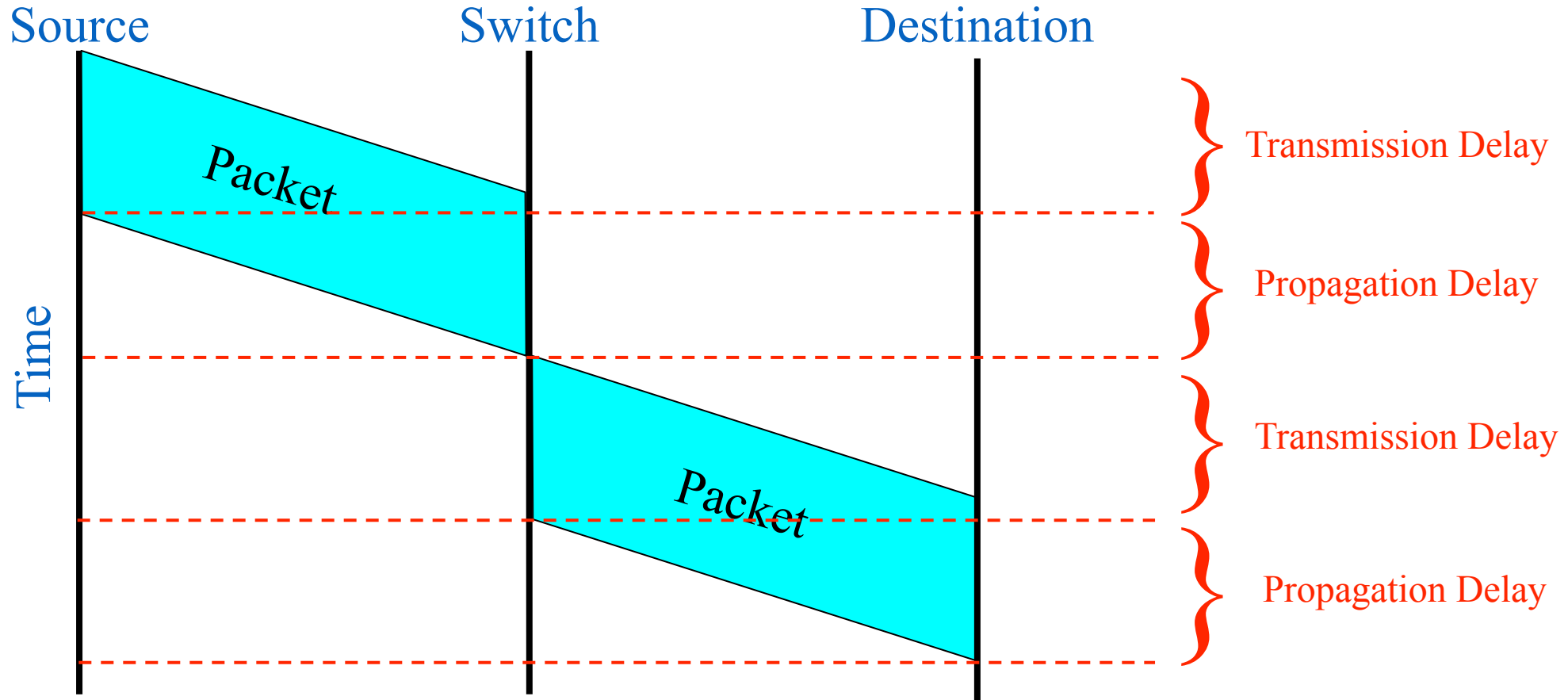
Performance Metrics - Q2

Calculate the time from first bit sent to last bit received for the following:

(a) You are sending a 5 Kb packet over a 10 Mbps link. There is a **store-and-forward switch** in the middle of the link, which begins retransmitting the packet as soon as it has finished receiving the packet. Assume each section of the link (source → switch, switch → destination) introduces a propagation delay of 100 microseconds.

$$\begin{aligned}\text{Total Time} &= 2 * (\text{Transmission} + \text{Propagation}) \\ &= 2 * ((5\text{Kb} / 10\text{Mbps}) + 1.0 * 10^{-4} \text{ s}) \\ &= 2 * (5.0 * 10^{-4} \text{ s} + 1.0 * 10^{-4} \text{ s}) \\ &= 1.2 * 10^{-3} \text{ seconds} = 1.2 \text{ ms}\end{aligned}$$

Performance Metrics - Q2



Performance Metrics - Q2

Calculate the latency (from first bit sent to last bit received) for the following:

(b) A 10-Mbps link with a single cut-through switch in the path, and a packet size of 5,000 bits. Assume that each section of the link introduces a propagation delay of 100 microseconds, and that the switch begins retransmitting immediately after the first 200 bits have been received.

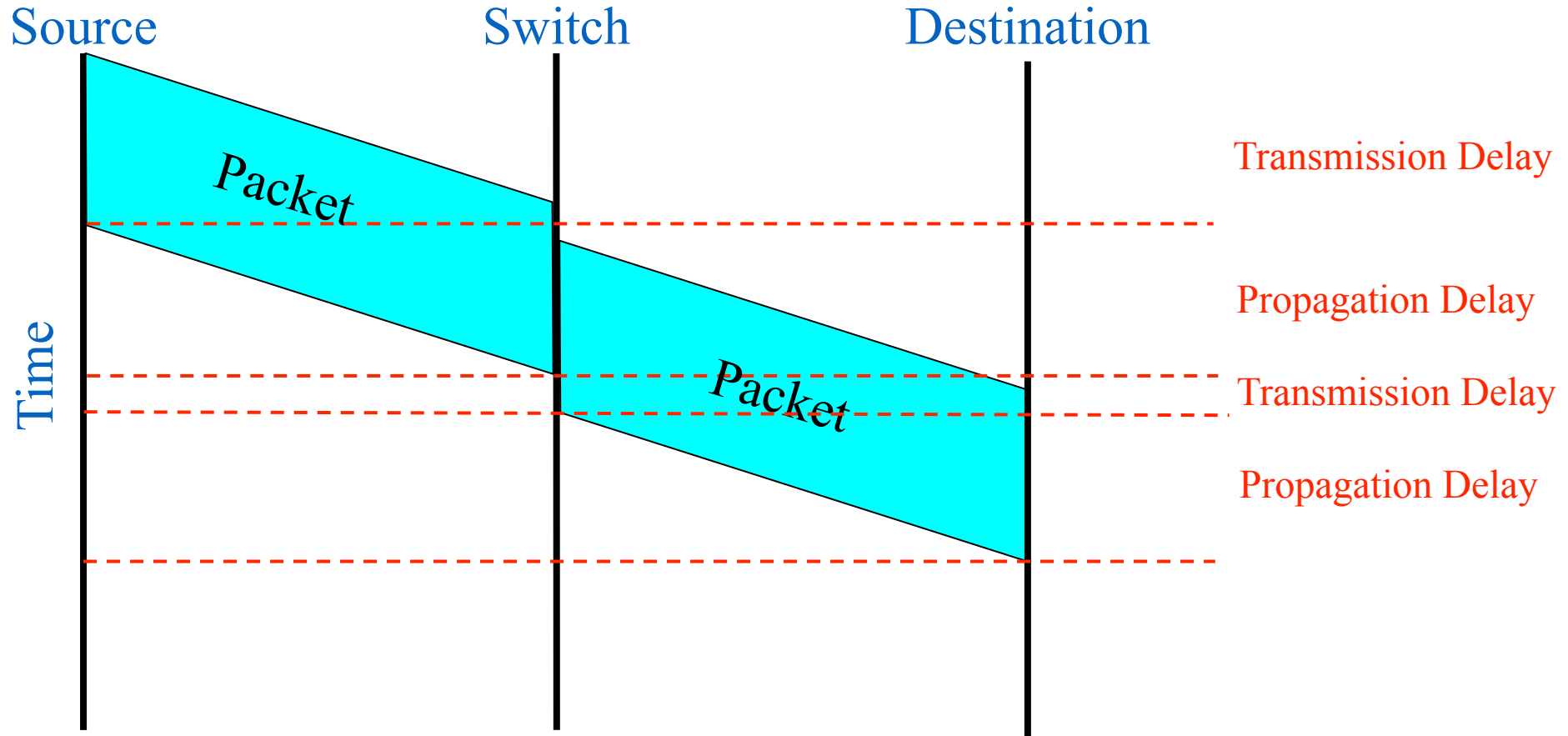
Performance Metrics - Q2

Calculate the latency (from first bit sent to last bit received) for the following:

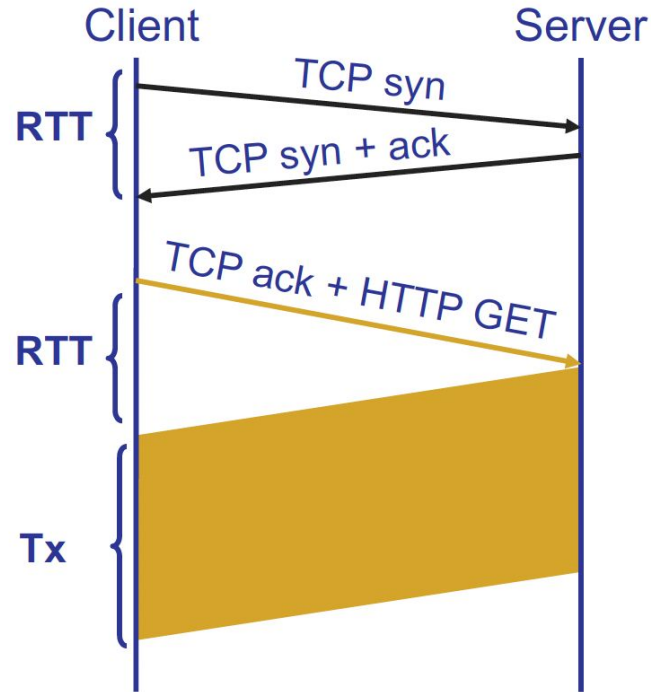
(b) A 10-Mbps link with a single cut-through switch in the path, and a packet size of 5,000 bits. Assume that each section of the link introduces a propagation delay of 100 microseconds, and that the switch begins retransmitting immediately after the first 200 bits have been received.

$$\begin{aligned}\text{Total Time} &= (\text{Transmission} + \text{Propagation}) + (\text{Transmission for 200 bits} + \text{Propagation}) \\ &= (5 \text{ Kb} / 10 \text{ Mbps}) + (1.0 * 10^{-4} \text{ s}) + (200 / 10 \text{ Mbps}) + (1.0 * 10^{-4} \text{ s}) \\ &= (5.0 * 10^{-4} \text{ s}) + (1.0 * 10^{-4} \text{ s}) + (0.2 * 10^{-4} \text{ s}) + (1.0 * 10^{-4} \text{ s}) \\ &= \mathbf{0.72 \text{ ms}}\end{aligned}$$

Performance Metrics - Q2



Performance Metrics - Q3

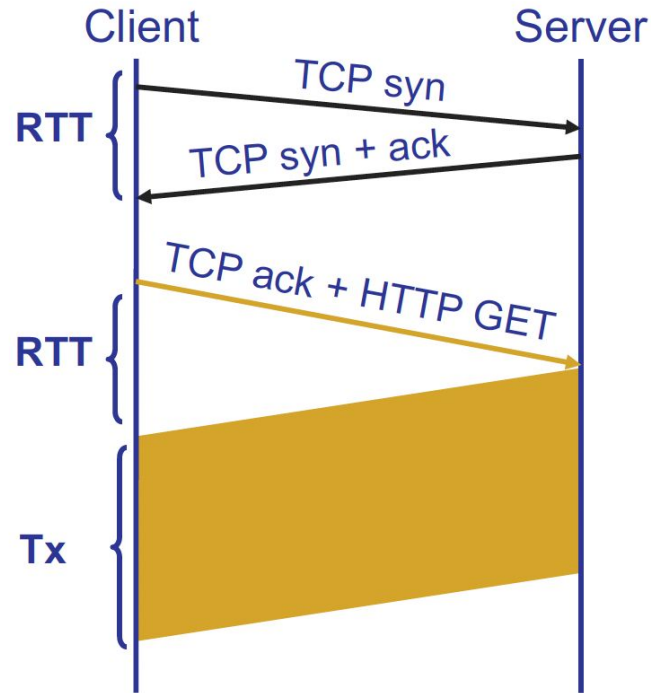


What is the object request response time if propagation delay = 0.15ms, transmission delay = 0.5ms?

Note: We ignore the transmission delay for the TCP syn/ack because the size of those messages is negligible compared to the size of the object, so the transmission delay can be treated as negligible.

Note: Ignore the gap on the client side between “TCP syn + ack” and “TCP ack + HTTP GET” – this is just a flawed diagram.

Performance Metrics - Q3



What is the object request response time if propagation delay = 0.15ms, transmission delay = 0.5ms?

$$\text{RTT} = 2 * \text{propagation delay} = 0.3\text{ms}$$

$$\text{Total} = 2 * \text{RTT} + \text{transmission delay} = \mathbf{1.1\text{ms}}$$

HTTP Protocol

HTTP 1.1 Request

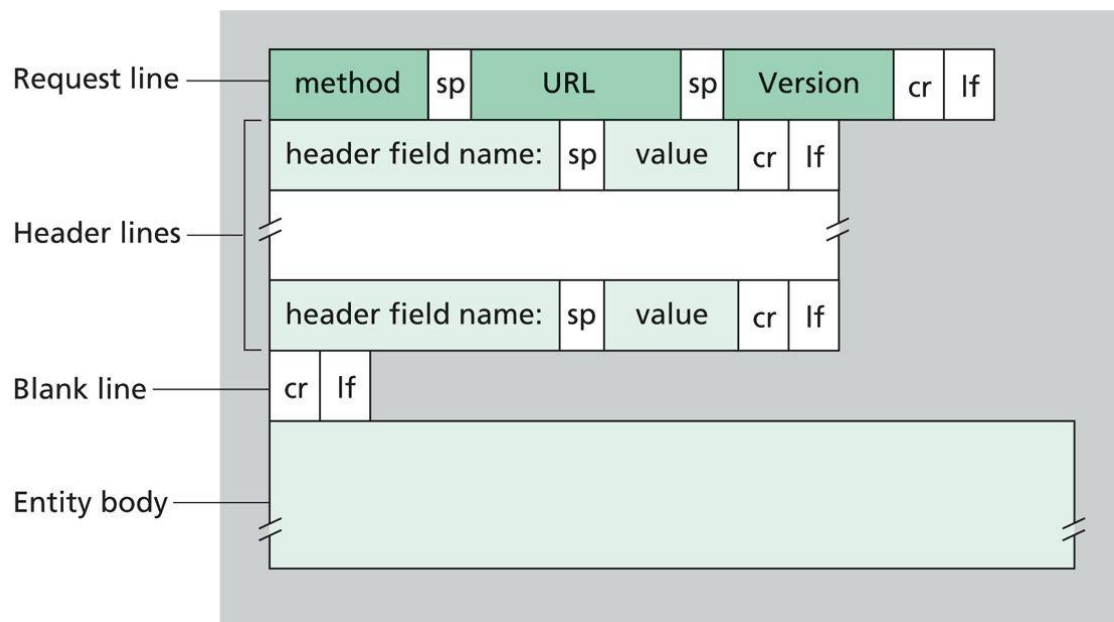
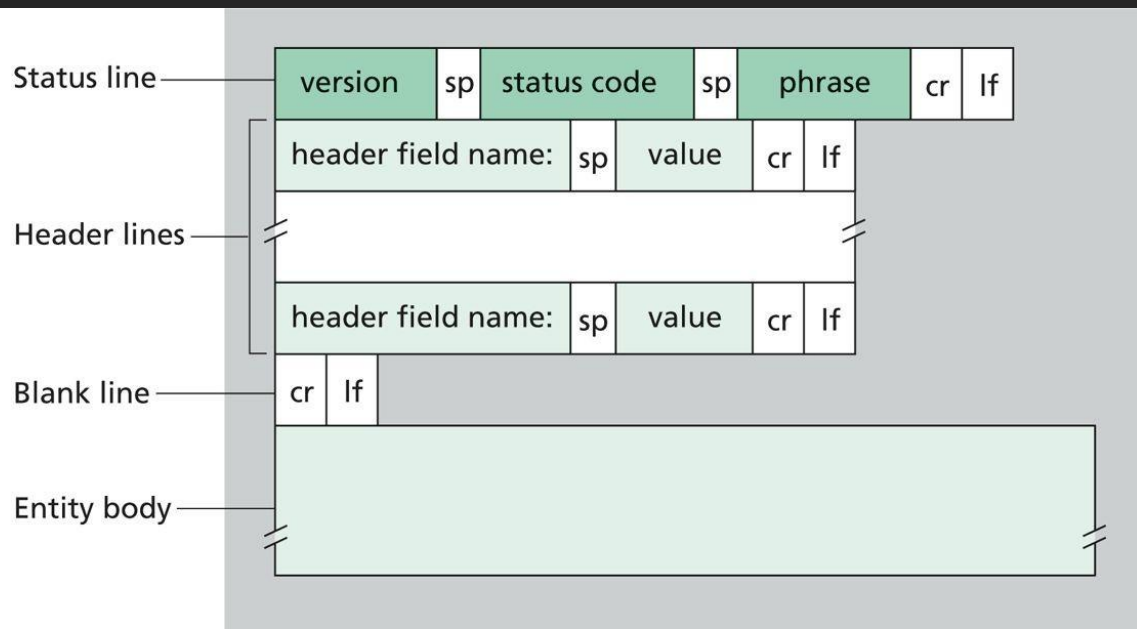


Figure 2.8 ♦ General format of a request message



HTTP 1.1 Response



HTTP/1.1 200 OK

Date: Sun, 08 Feb xxxx 01:11:12 GMT

Server: Apache/1.3.29 (Win32)

Last-Modified: Sat, 07 Feb xxxx

ETag: "0-23-4024c3a5"

Accept-Ranges: bytes

Content-Length: 35

Connection: close

Content-Type: text/html

<h1>My Home page</h1>

Figure 2.9 ♦ General format of a response message

Status Codes

The first digit of the Status-Code defines the class of response. The last two digits do not have any categorization role. There are 5 values for the first digit:

- 1xx: Informational - Request received, continuing process
- 2xx: Success - The action was successfully received, understood, and accepted
- 3xx: Redirection - Further action must be taken in order to complete the request
- 4xx: Client Error - The request contains bad syntax or cannot be fulfilled
- 5xx: Server Error - The server failed to fulfill an apparently valid request

A note on CR/LF



- **CR:** Carriage Return (**0x0D**) – often `\r`
- **LF:** Line Feed (**0x0A**) – often `\n`
- Windows traditionally uses CR LF, Unix uses just LF to break lines apart – will often see `\r\n` when printing out HTTP requests.
- Comes from typewriters – **LF** moves the paper up and **CR** returns the cursor to the leftmost character of the line.

Q1

True or False: HTTP 1.1 response messages never have an empty message body.

Q1

True or False: HTTP 1.1 response messages never have an empty message body.

False. Some HTTP response messages have an empty message body.

- E.g. A HEAD request does not require (and usually doesn't have) a message body.
- E.g. A HEAD response **may not have** a message body.

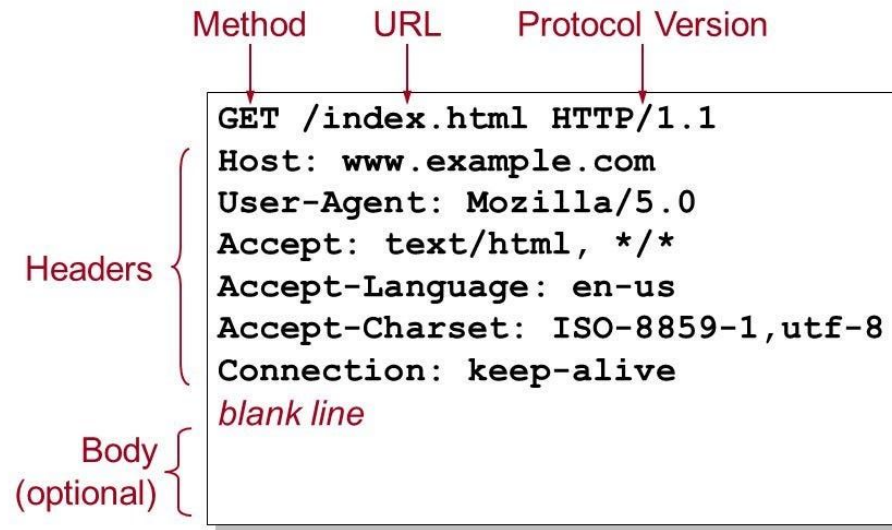
HEAD

The HTTP **HEAD** method requests the headers that would be returned if the **HEAD** request's URL was instead requested with the HTTP GET method. For example, if a URL might produce a large download, a **HEAD** request could read its Content-Length header to check the filesize without actually downloading the file.

Q2

True or False: HTTP 1.1 request messages never have an empty set of headers.

HTTP Request



Q2

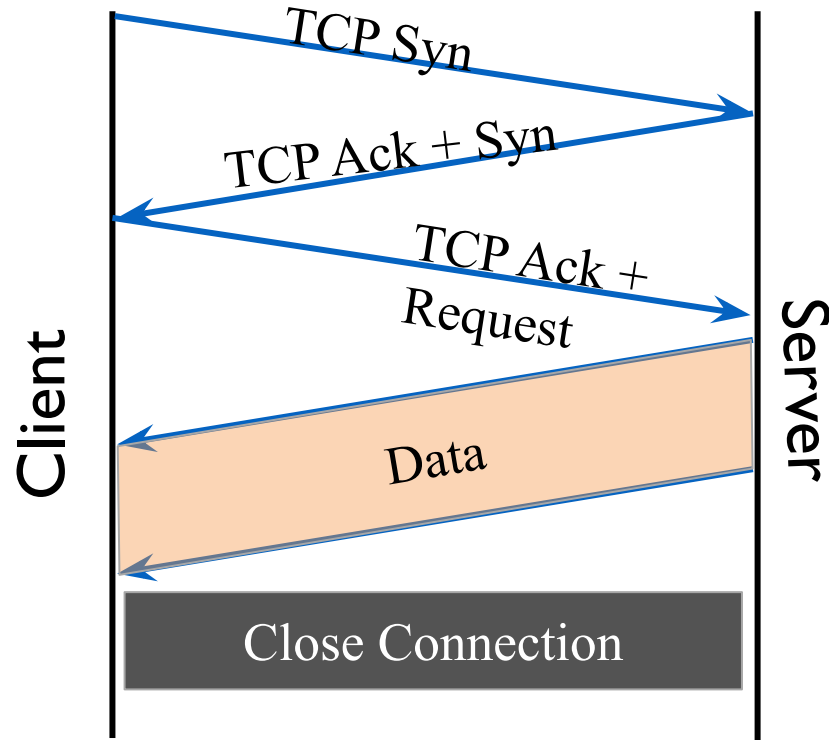
True or False: HTTP 1.1 request messages never have an empty set of headers.

True. For example, the **Host** header is required!

HTTP Request



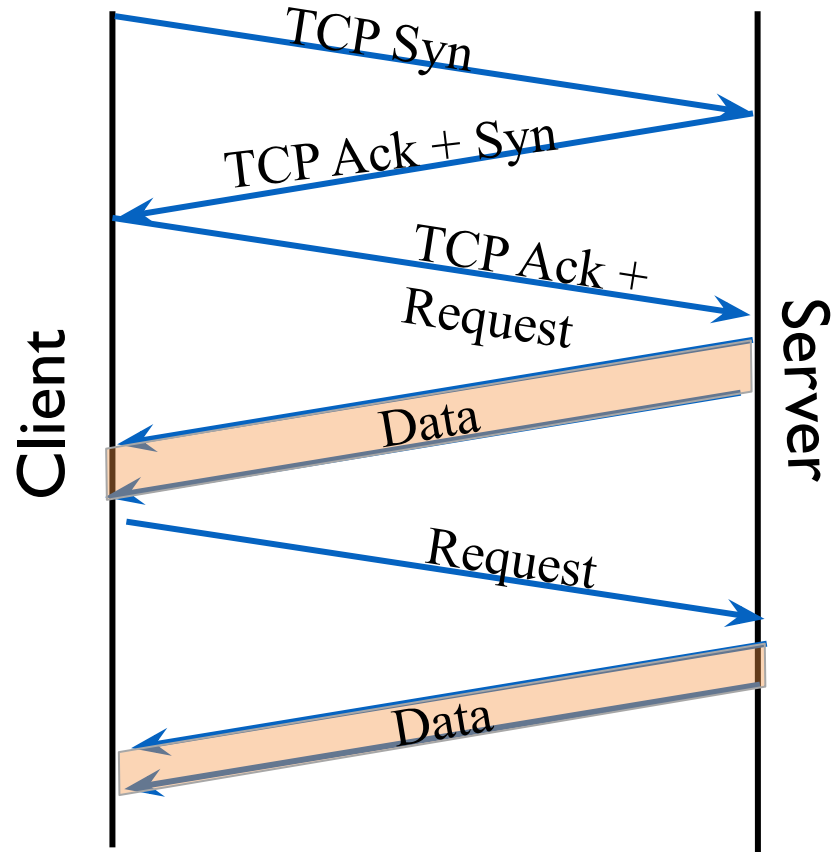
Persistency & Multiplexing



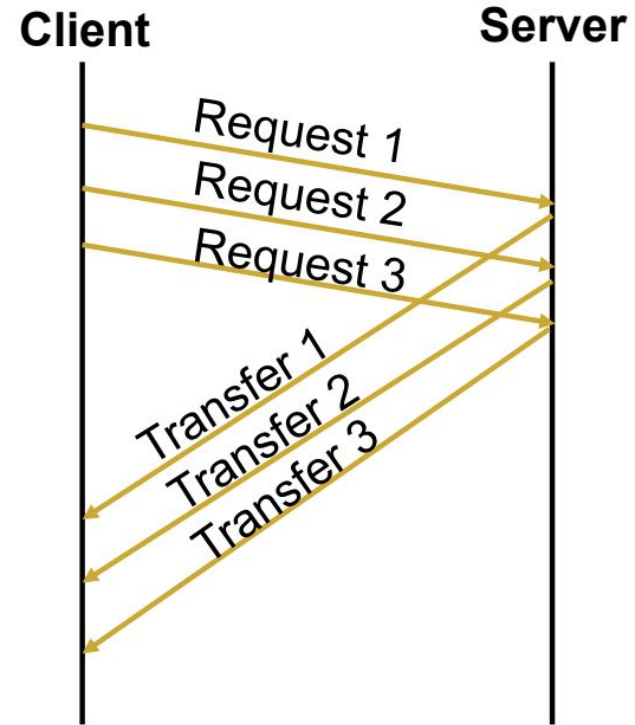
Non-Persistent Connection

- In **HTTP 1.0**, connections are non-persistent by default – they terminate after one piece of data is exchanged.
- In **HTTP 1.1**, connections are persistent by default, meaning that they aren't closed after one request.
- In **HTTP 2**, connections can be multiplexed, which means that multiple requests can be sent concurrently.

Persistency & Multiplexed Connection



Persistent Connection



Multiplexed Connection

Q3

True or False: Two distinct Web pages (for example, umich.edu/research.html and umich.edu/students.html) can be sent over the same persistent connection.

Q3

True or False: Two distinct Web pages (for example, umich.edu/research.html and umich.edu/students.html) can be sent over the same persistent connection.

True. Both these pages are on the same physical server, so they can be retrieved on the same persistent connection.

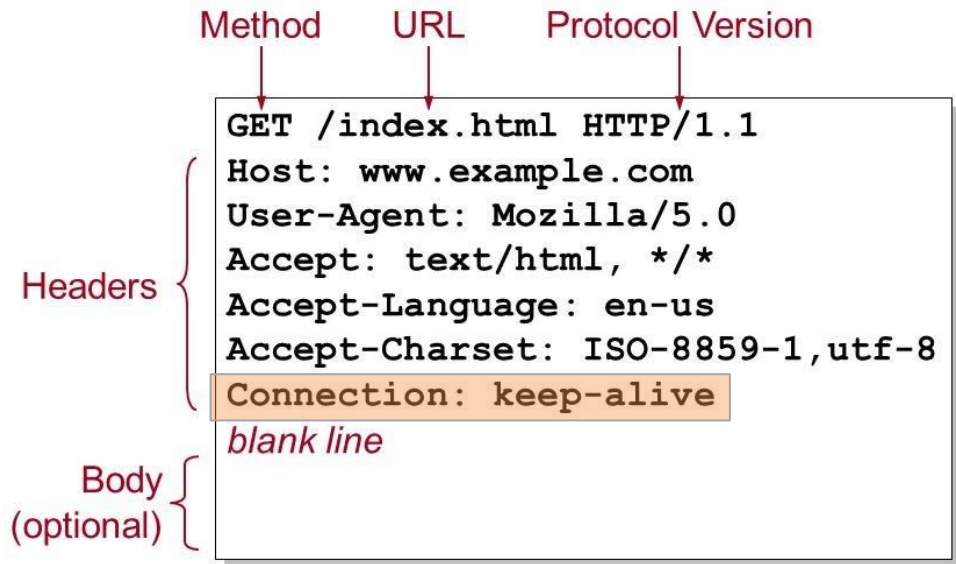
Q4

True or False: Any requests sent over HTTP 2 must be persistent.

Q4

True or False: Any requests sent over HTTP 2 must be persistent.

False. The persistency of the request is controlled by the `Connection` header, which is `keep-alive` by default (for a persistent connection), but can be set to `close` for a non-persistent connection.



Q5

True or False: Pipelining and multiplexing refer to the same thing.

Q5

True or False: Pipelining and multiplexing refer to the same thing.

False.

Pipelining

- Introduced in HTML 1.1, but not supported in most implementations of HTML 1.1.
- Allows for clients to make multiple requests at once.
- Servers must respond to requests in-order.
- Does not solve head-of-line blocking problem.

Multiplexing

- Introduced in HTML 2.
- Allows for clients to make multiple requests at once.
- Servers can respond to requests in any order they want.
- Solves head-of-line blocking problem!

Q6

You request a very small HTML file from a server. This HTML references **eight** other very small images. Let **X** denote the RTT between the localhost and the server. How much time elapses with **non-persistent HTTP with no parallel TCP connections**?

Q6

You request a very small HTML file from a server. This HTML references **eight** other very small images. Let **X** denote the RTT between the localhost and the server. How much time elapses with **non-persistent HTTP with no parallel TCP connections**?

$$2X + 8 * (2X) = 18X$$

Q7

You request a very small HTML file from a server. This HTML references **eight** other very small images. Let **X** denote the RTT between the localhost and the server. How much time elapses with **non-persistent HTTP with the browser configured for 5 parallel connections?**

Q7

You request a very small HTML file from a server. This HTML references **eight** other very small images. Let **X** denote the RTT between the localhost and the server. How much time elapses with **non-persistent HTTP with the browser configured for 5 parallel connections?**

$$2X + \text{CEIL}(8 / 5) * (2X) = 2X + 2 * 2X = 6X$$

Q8

You request a very small HTML file from a server. This HTML references eight other very small images. Let X denote the RTT between the localhost and the server. How much time elapses with **persistent HTTP with multiplexing**?

Q8

You request a very small HTML file from a server. This HTML references eight other very small images. Let X denote the RTT between the localhost and the server. How much time elapses with **persistent HTTP with multiplexing**?

$$X + 2X = 3X$$

